pyGWB and stochastic.m comparison

Guo Chin, Shivaraj Kandhasamy, Joe Romano

Cross-correlation statistics

arXiv:1410.6211

- If d1 and d2 are detector data, the signal statistic Ω_α is defined as

$$\hat{\Omega}_\alpha(f) = \frac{2}{T}\frac{\Re\left[\tilde{d}_1^*(f)\tilde{d}_2(f)\right]}{\gamma_{12}(f)S_\alpha(f)}$$

where

$$S_\alpha(f) = \frac{3H^2}{10\pi^2}\frac{1}{f^3}\left(\frac{f}{f_{ref}}\right)^\alpha$$

and its variance by

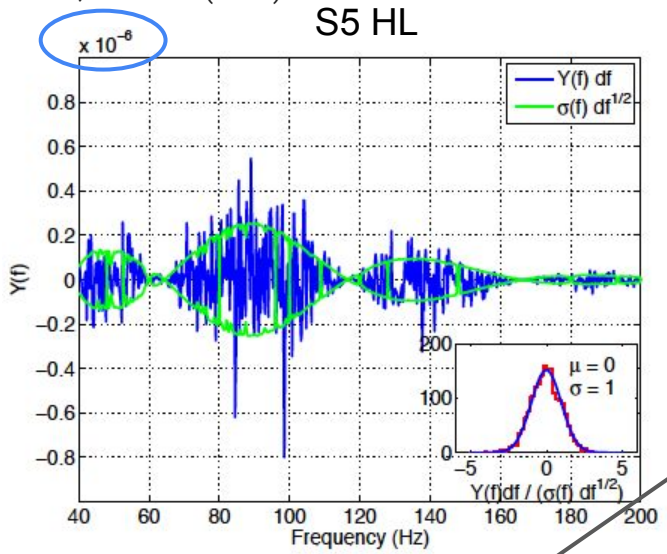
$$\sigma^2_{\hat{\Omega}_\alpha}(f) \approx \frac{1}{2T\Delta f}\frac{P_1(f)P_2(f)}{\gamma_{12}^2(f)S_\alpha^2(f)}$$

- The final estimate of Ω_α and its variance is obtained by the weighted sum of estimates at different frequencies

$$\hat{\Omega}_\alpha \equiv \frac{\sum_{t,f}\sigma^{-2}_{\hat{\Omega}_\alpha}(f)\hat{\Omega}_\alpha(f)}{\sum_{t,f}\sigma^{-2}_{\hat{\Omega}_\alpha}(f)} \qquad \sigma^2_{\hat{\Omega}_\alpha} \equiv \left[\sum_{t,f}\sigma^{-2}_{\hat{\Omega}_\alpha}(f)\right]^{-1}$$

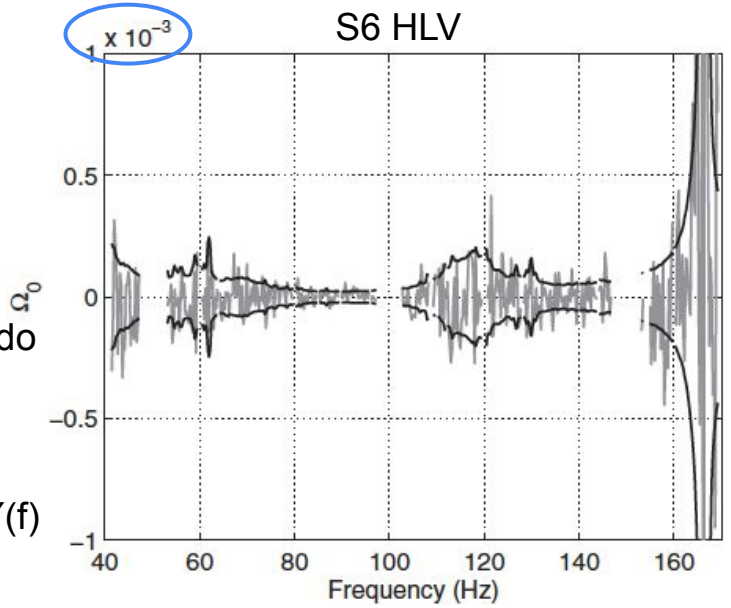
Cross-correlation statistics

Nature 460, 990–994 (2009)



If we want to do additional notching, we need to renormalize Y(f)

Phys. Rev. Lett. **113**, 231101 (2014)



$$Y = \int_0^{+\infty} df\, Y(f)$$

calculated in the post-processing

$\Omega_\alpha(f)$

$$\hat{\Omega}_0 = \frac{\sum_f \sigma_{\hat{\Omega}_0}^{-2}(f)\hat{\Omega}_0(f)}{\sum_f \sigma_{\hat{\Omega}_0}^{-2}(f)}$$

Currently stochastic.m calculates

- In pyGWB we will directly calculate $\Omega_\alpha(f)$

Cross-correlation statistics

- The power spectral densities and cross-spectral densities of the detectors are defined as,

$$P_1(f) = \frac{2}{T}\,\tilde{d}_1^*(f)\,\tilde{d}_1(f) \qquad P_2(f) = \frac{2}{T}\,\tilde{d}_2^*(f)\,\tilde{d}_2(f)$$

$$C(f) = \frac{2}{T}\,\tilde{d}_1^*(f)\,\tilde{d}_2(f)$$

- Re-writing Ω_α in terms of C(f), we get,

$$\hat{\Omega}_\alpha(f) = \frac{\Re\left[C(f)\right]}{\gamma_{12}(f) S_\alpha(f)}$$

pyGWB convention

- FFTs returned by *gwpy fftgram* (scipy.signal.spectrogram) in pyGWB is one-sided amplitude spectral densities such that,

$$P_{1,2}(f) = 2\left[\mathrm{FFT}(d_{1,2})\right]^* \times \mathrm{FFT}(d_{1,2})$$

$$C(f) = 2\left[\mathrm{FFT}(d_1)\right]^* \times \mathrm{FFT}(d_2)$$

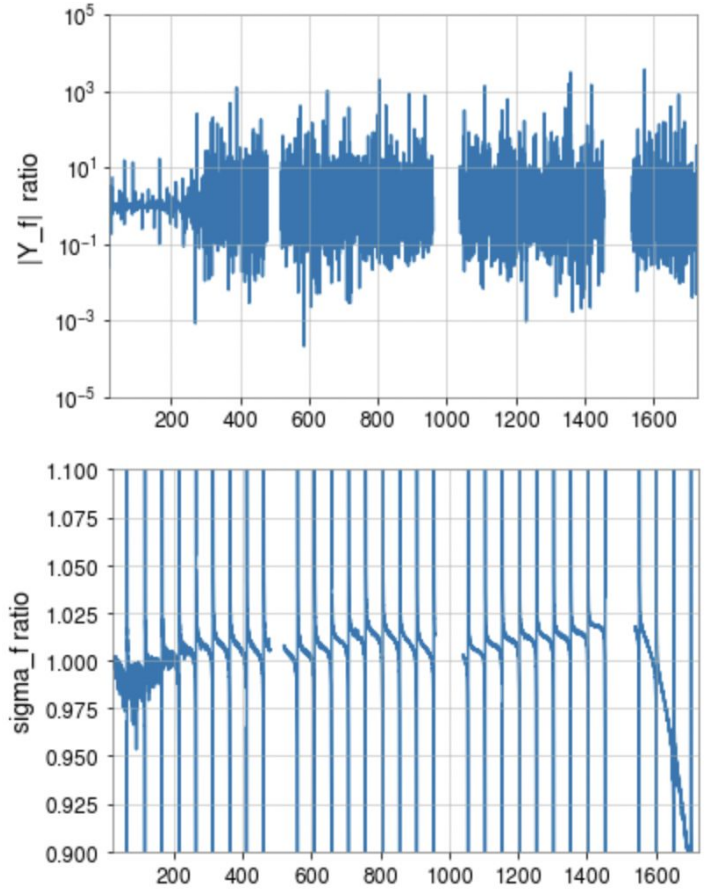
- The above FFTs also include scale factors needed to account for (non-rectangular) windows. So we don’t need to add window factors explicitly for C(f) as we currently do.
  - But we still have to undo the window factors in the estimation of $P_{1,2}$

Comparison with stochastic.m

- A single job from O3 that produces three overlapping segments was used for the comparison.
  - 1247644138 - 1247645038
  - Channel: {H1,L1}: GWOSC-16KHZ_R1_STRAIN
  - O3 frequency notching was applied; no badGPSTimes cut
  - It was randomly chosen; it turns out there was a glitch in H1 in the first segment

Comparison with stochastic.m

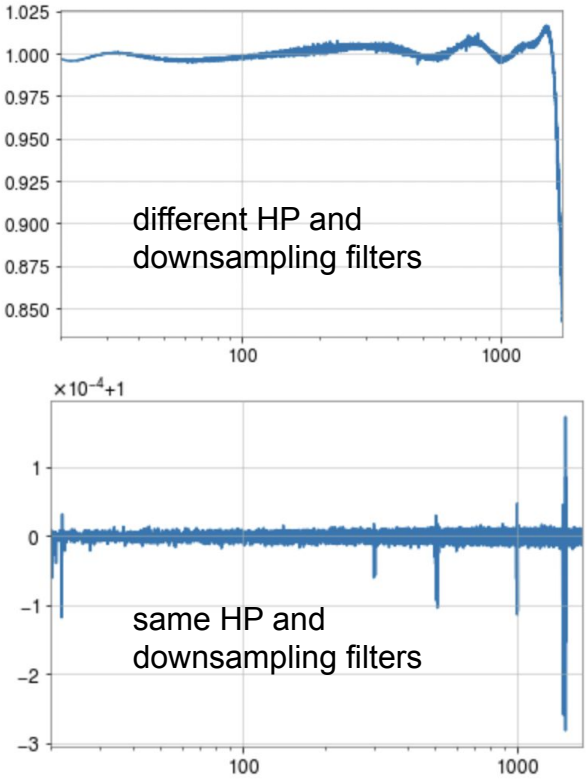
```
Comparing point estimates
        stochastic.m: -2.974485e-06
        pyGWB: -2.932505e-06
        % diff: 1.411359%
Comparing sigmas
        stochastic.m: 1.231366e-06
        pyGWB: 1.226274e-06
        % diff: 0.413484%
```



Comparison with stochastic.m

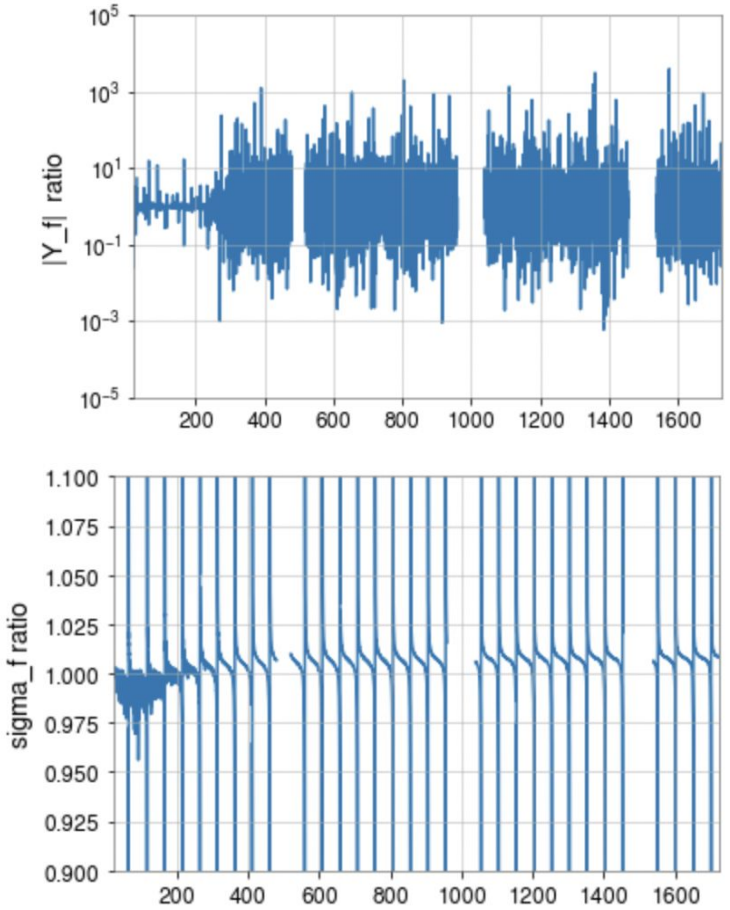
- Some of the differences in point estimate and sigma between matlab and pyGWB comes from differences in High-Pass(HP) filter and downsampling filters used
- So for tests we used HP filtered and downsampled data from matlab and imported in pyGWB

Ratio of PSDs estimated in matlab/pyGWB



Comparison with stochastic.m

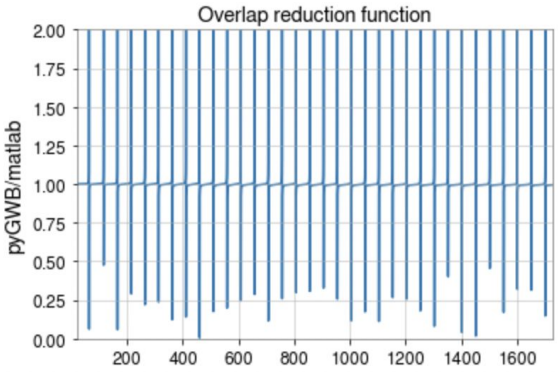
```
Comparing point estimates
        stochastic.m: -2.974485e-06
        pyGWB: -2.937422e-06
        % diff: 1.246034%
Comparing sigmas
        stochastic.m: 1.231366e-06
        pyGWB: 1.227871e-06
        % diff: 0.283786%
```

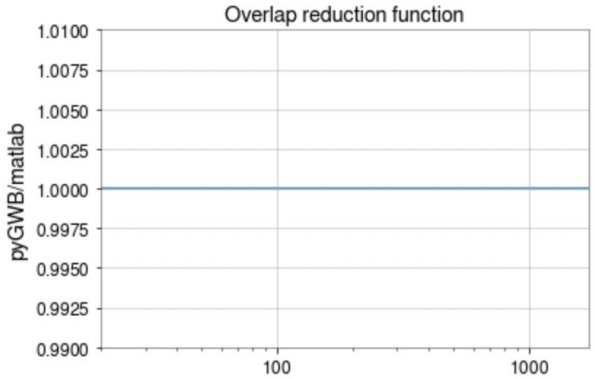


Comparison with stochastic.m

- There are small differences in the ORF, mainly near the zeros of ORF (may be there is a slight offset between two ORFs or something like that). To eliminate this as source of difference, we also used the ORF from matlab.
- The differences in ORF could be from some kind of precision issue between stochastic and pyGWB. May be something to check out.

Ratio of ORF estimated in matlab/pyGWB

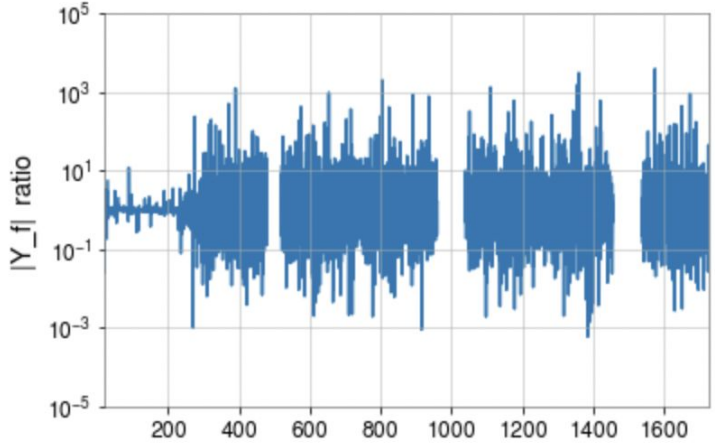


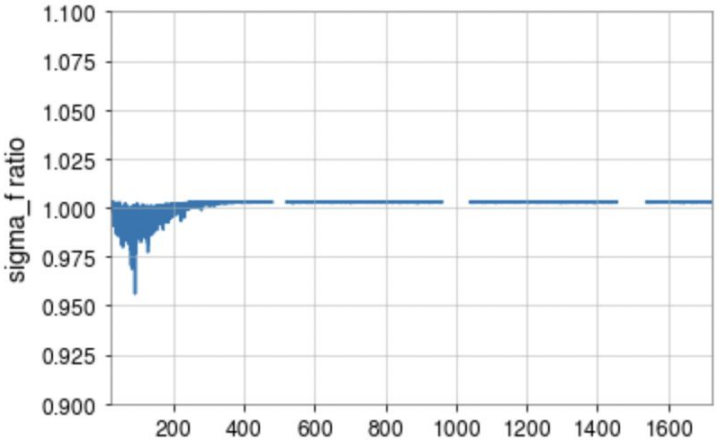


Comparison with stochastic.m

```
Comparing point estimates
        stochastic.m: -2.974485e-06
        pyGWB: -2.939829e-06
        % diff: 1.165122%
Comparing sigmas
        stochastic.m: 1.231366e-06
        pyGWB: 1.228510e-06
        % diff: 0.231930%

Final point estimate  = -2.974485e-06
Final error bar   = 1.172285e-06
Number segs = 3
```



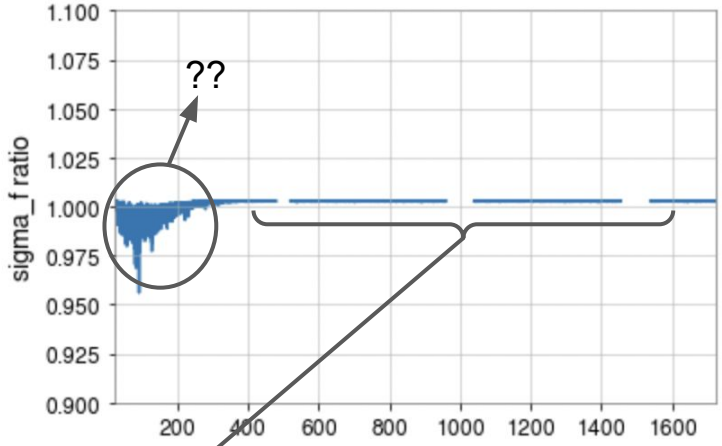


Comparison with stochastic.m

```
Comparing point estimates
        stochastic.m: -2.974485e-06
        pyGWB: -2.939829e-06
        % diff: 1.165122%
Comparing sigmas
        stochastic.m: 1.231366e-06
        pyGWB: 1.228510e-06
        % diff: 0.231930%
```

Seems a bit large

- Individual segment-wise sigmas between pyGWB and matlab agrees very well (0.9999993014205865, 1.0000030412497258, 1.0000030349600166)
- Need to check if there are any differences in how the data is combined in pyGWB and matlab.



Good agreement expected

Agrees to ~ floating point precision

$$\sigma^2_{\hat{\Omega}_\alpha}(f) \approx \frac{1}{2T\Delta f}\frac{P_1(f)P_2(f)}{\gamma^2_{12}(f)S^2_\alpha(f)}$$

identical between matlab and pyGWB

constant

Point estimate comparison

- Overall the point estimate spectra between matlab and pyGWB indeed tracks each other. But there are small differences as we see in the plots. It might be coming from precision issues.
  - But the issue may not be between matlab and python, but how Y(f) is calculated in two methods even within matlab.

$$Y = \int_0^{+\infty} df\, Y(f)$$

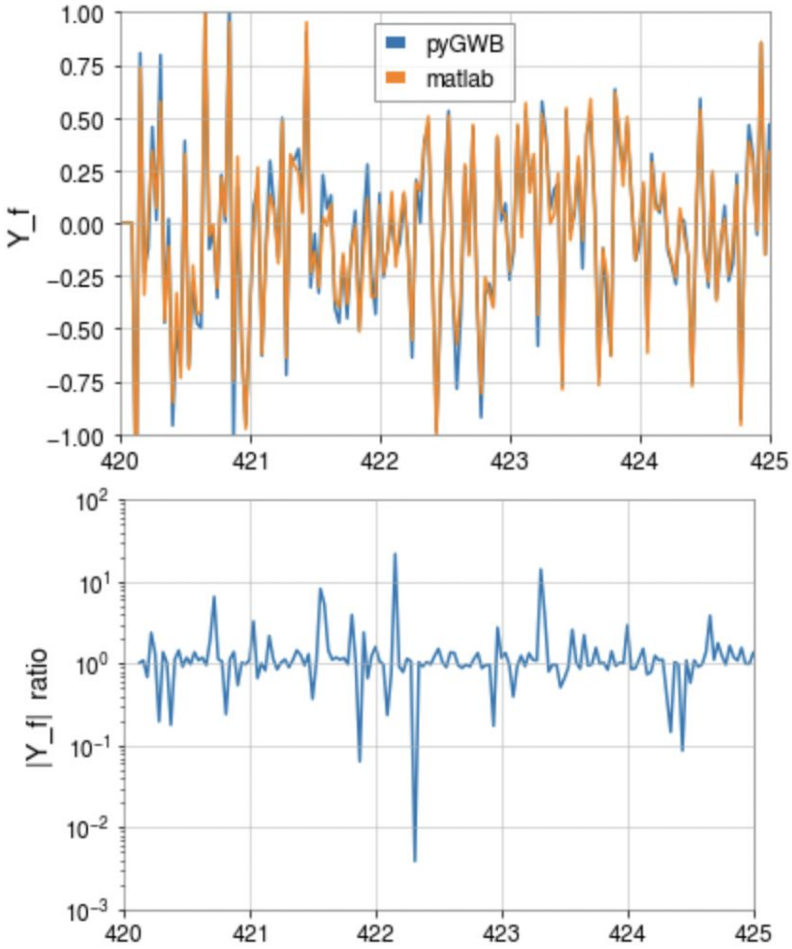
for each segment

$$\Omega_\alpha, \sigma_{\Omega_\alpha}$$

Combine segments

Final Y(f),\sigma(f)

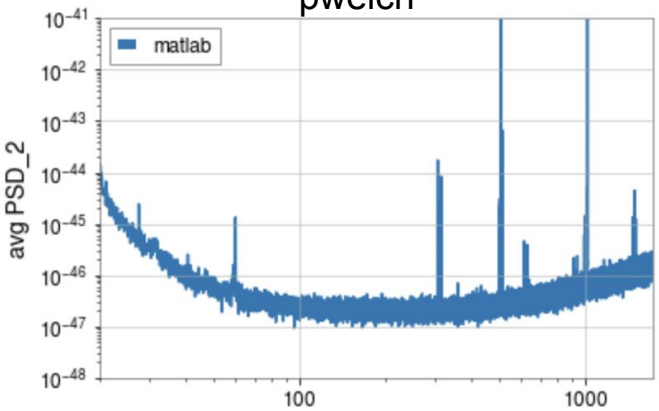
$$\hat{\Omega}_\alpha \equiv \frac{\sum_{t,f} \sigma_{\hat{\Omega}_\alpha}^{-2}(f)\hat{\Omega}_\alpha(f)}{\sum_{t,f} \sigma_{\hat{\Omega}_\alpha}^{-2}(f)}$$



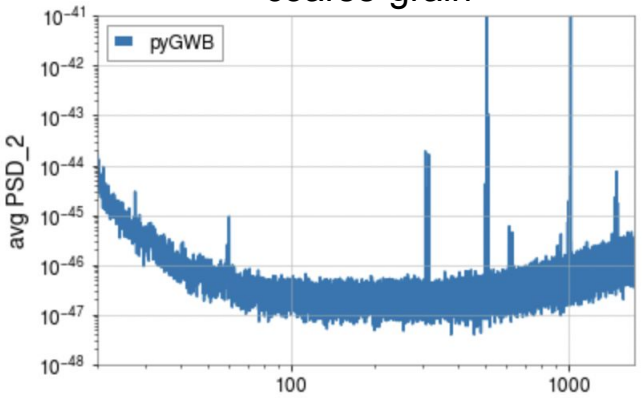
A couple of points

- PSD estimation (pwelch vs coarse-graining)
  - So far we have been using pwelch time-average method for estimating PSDs
  - There is proposal for using coarse-graining for PSD estimation in pyGWB
  - The variance of the PSD returned by coarse-graining is larger than the PSD from welch estimate.
    - From an estimate point view, this suggests use of pwelch method might be better
  - All the comparison shown in this presentation were using pwelch time-average

pwelch



coarse-grain



A couple of points

1. Numerical recipes, the art of scientific computing : 13.1 Convolution and Deconvolution Using the FFT
2. DCC T040125 - Cross-correlation of windowed, discretely-sampled data

- Zero-padding (and thus coarse-graining for CSD).
  - If the statistics is defined in the time domain, for example as below,

$$\Omega = \sum_{i}^{N-1} \delta t \sum_{j}^{N-1} \delta t \, d_1[j] \, Q[j-k] \, d_2[k]$$

    where $d_{1,2}$, Q the optimal filter.

    Then the expression in the frequency domain we saw in the first slide arises as a result of convolution theorem. However this assumes that the data is periodic. This corrupts the the points at the beginning and end of the convolved data. Zero padding the data (d) equal to the half the size of Q solves this problem (in this case Q is of length 2N or twice the length of d's).
- For all the results shown in this presentation, we have used zero-padding in CSD estimation

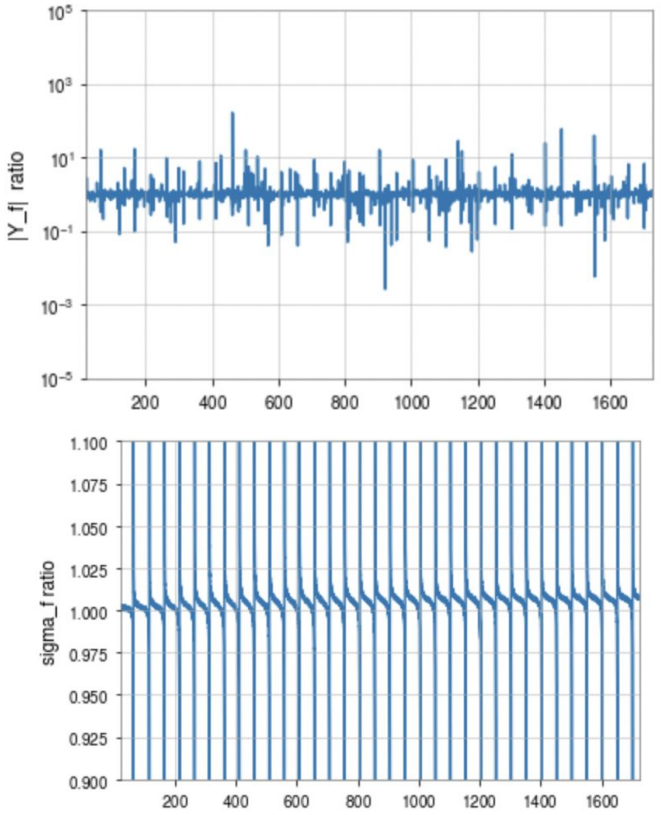
Comparison using O2 data set

- Here we compare stochastic.m and pyGWB using O2 data set used in stochastic_lite comparison

```
Comparing point estimates
        stochastic.m: 5.954429e-07
        pyGWB: 5.984278e-07
        % diff: 0.501284%
Comparing sigmas
        stochastic.m: 1.939605e-06
        pyGWB: 1.939246e-06
        % diff: 0.018532%
```

```
Comparing point estimates
        stochastic.m: 5.954429e-07
        stochastic_lite: 5.984385e-07
        % diff: 0.500556%
Comparing sigmas
        stochastic.m: 1.939605e-06
        stochastic_lite: 1.942621e-06
        % diff: 0.155230%
```

Similar to stochastic_lite plot



Future work

- The notebook used for these tests is available at,

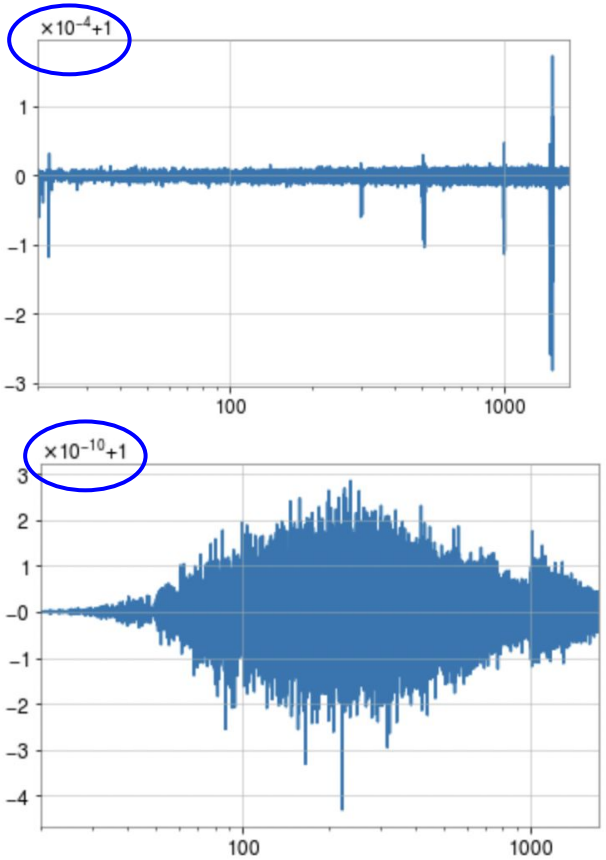
  https://git.ligo.org/shivaraj.kandhasamy/pygwb/-/blob/master/tutorials/integrationTest_O3_data.ipynb

  It is also added as a merge request in the main pygwb repository.

- We will work on resolving the differences in post-processing step.
- In this test we have used (at least imported in the notebook) network, pre_processing, spectral, post_processing, util modules.
- We haven't used data quality, statistical checks, PE and simulated data modules.
- It might be good to start thinking on the workflow for a small set of jobs for a single baseline.
    - If others use this notebook for such tests, it will help improve the readability of the notebook.

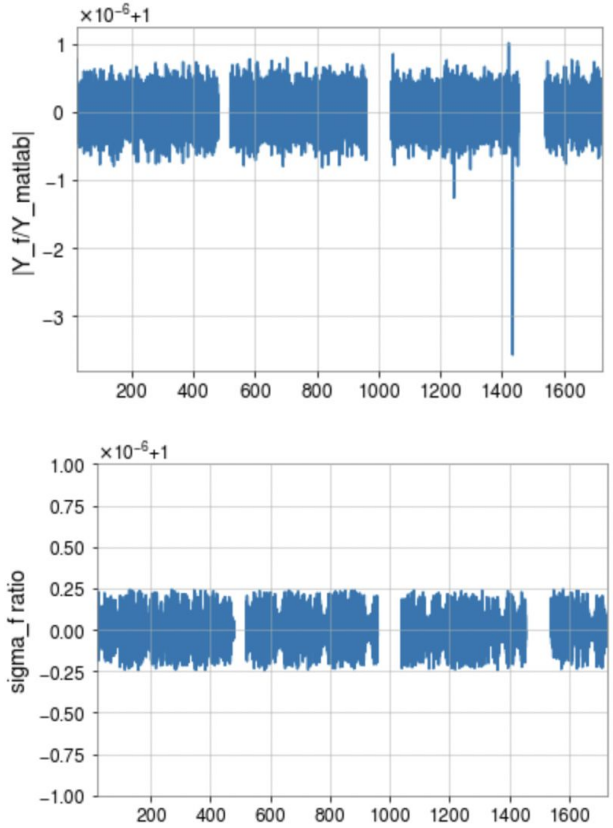
Comparison using non-overlapping segments

- Difference from using non-symmetric window
  - Default by singal.get_window() function.



Comparison using non-overlapping segments

```
Comparing point estimates
        stochastic.m: -3.117905e-06
        pyGWB: -3.117904e-06
        % diff: 0.000028%
Comparing sigmas
        stochastic.m: 1.246792e-06
        pyGWB: 1.246792e-06
        % diff: 0.000000%
```



Expressions for final \sigma in stochastic.m

$$\sigma_J^{-2} = \left[\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - k\left(\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - \frac{1}{2}(\sigma_{1J}^{-2} + \sigma_{NJ}^{-2})\right)\right] \times \frac{1}{1 - \frac{k^2}{4}\sigma_{oo}^2\sigma_{ee}^2\left(\sigma_{oo}^{-2} + \sigma_{ee}^{-2} - \frac{1}{2}(\sigma_1^{-2} + \sigma_N^{-2})\right)^2}$$

$$k = \frac{\overline{w^2 w_{ovl}^2}}{\overline{w^2 w^2}}$$

$$\sigma_{oo,J}^{-2} = \sum_m \sigma_{mJ}^{-2} \qquad \sigma_{ee,J}^{-2} = \sum_n \sigma_{nJ}^{-2} \qquad \begin{matrix} m = 1, 3, 5, \ldots \\ n = 2, 4, 6, \ldots \end{matrix}$$

For 50% overlapping hann windows

$k = 0.0857$

$$\sigma_{oo}^{-2} = \sum_J \sigma_{oo,J}^{-2} = \sum_J \sum_m \sigma_{mJ}^{-2} \qquad \sigma_{ee}^{-2} = \sum_J \sigma_{ee,J}^{-2} = \sum_J \sum_n \sigma_{nJ}^{-2}$$

Expressions for final \sigma in pyGWB

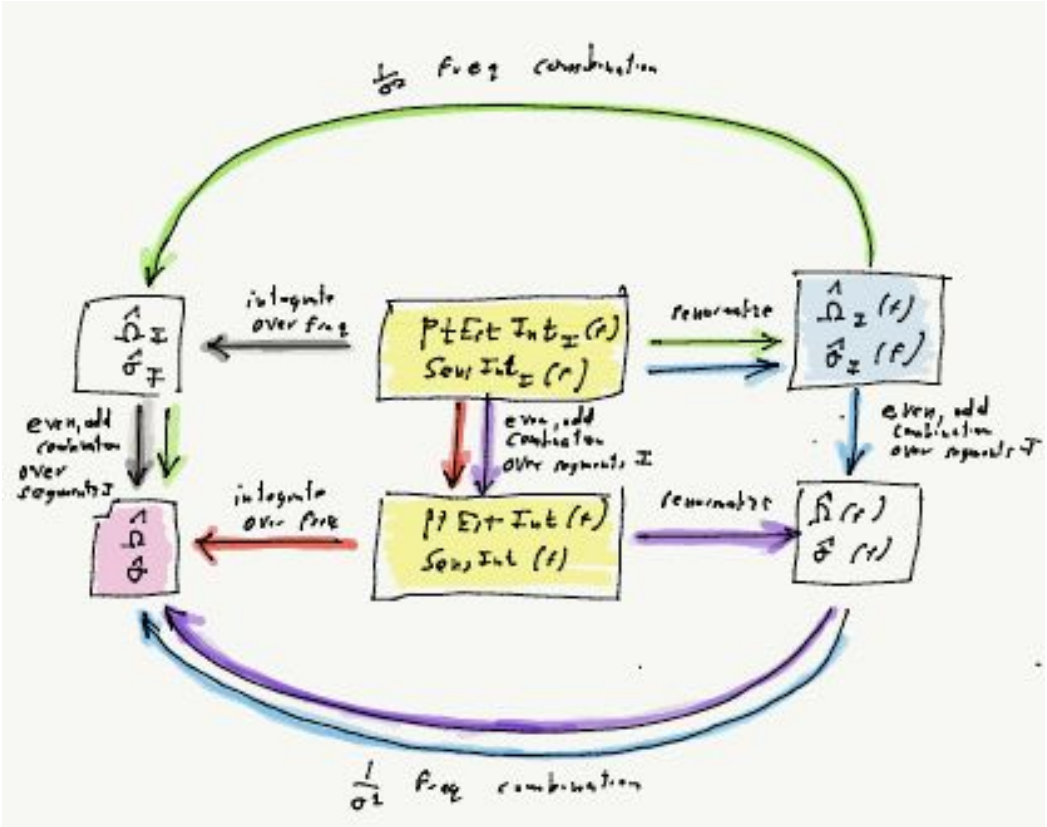
$$\sigma_J^{-2} = \left[\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - k\left(\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - \frac{1}{2}(\sigma_{1J}^{-2} + \sigma_{NJ}^{-2}\right)\right] \times \frac{1}{1 - \frac{k^2}{4}\,\sigma_{oo,J}^2\,\sigma_{ee,J}^2\left(\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - \frac{1}{2}(\sigma_{1J}^{-2} + \sigma_{NJ}^{-2})\right)^2}$$

- The above expression is what currently implemented in pyGWB that gives different results than stochastic.m

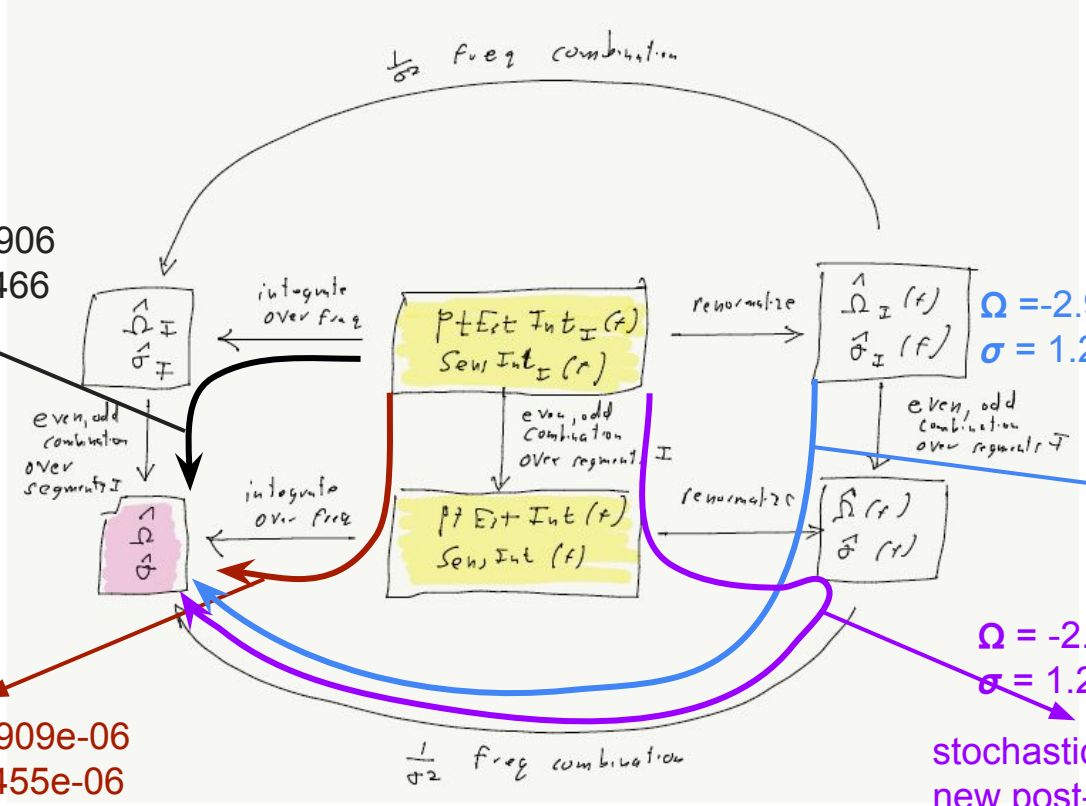
Expressions for final \Omega in stochastic.m

$$\Omega_J = \left(\Omega_{oo,J} \sigma_{oo,J}^{-2} \left[1 - \frac{k}{2} \sigma_{oo}^2 \left(\sigma_{oo}^{-2} + \sigma_{ee}^{-2} - \frac{1}{2} \left(\sigma_1^{-2} + \sigma_N^{-2} \right) \right) \right] + \right.$$
$$\left. \Omega_{ee,J} \sigma_{ee,J}^{-2} \left[1 - \frac{k}{2} \sigma_{ee}^2 \left(\sigma_{oo}^{-2} + \sigma_{ee}^{-2} - \frac{1}{2} \left(\sigma_1^{-2} + \sigma_N^{-2} \right) \right) \right] \right) /$$
$$\left(\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - k \left[\sigma_{oo,J}^{-2} + \sigma_{ee,J}^{-2} - \frac{1}{2} \left(\sigma_{1,J}^{-2} + \sigma_{N,J}^{-2} \right) \right] \right)$$

Estimators in stochastic.m and pyGWB



Estimators in stochastic.m and pyGWB



Ω =-2.974485389642906
σ = 1.231365695099466

Ω =-2.939808604647754e-06
σ = 1.2285101751924176e-06

pyGWB

Ω = -2.9744853555526576e-06
σ = 1.231365694433404e-06

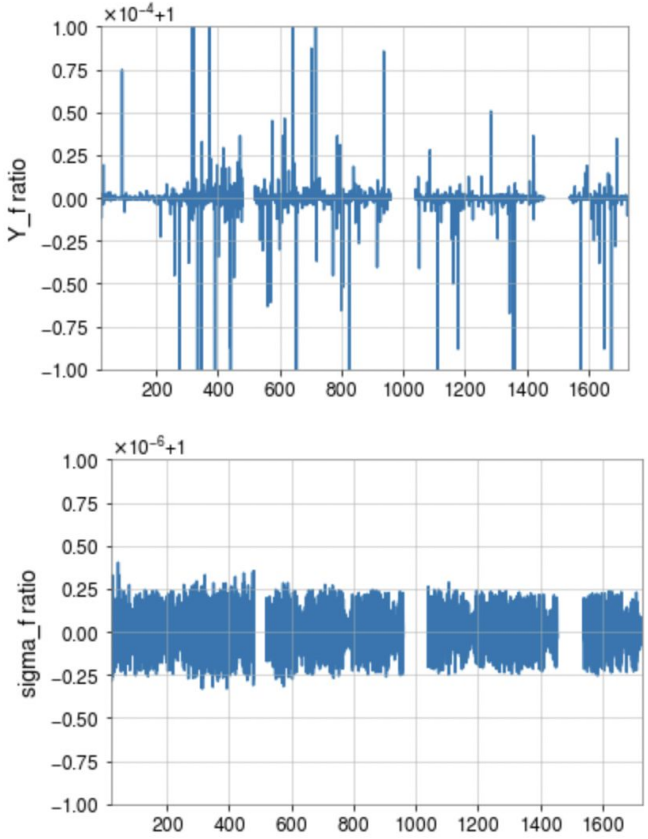
stochastic.m + new post-processing

stochastic.m

Ω =-2.974485389642909e-06
σ = 1.231365695099455e-06

Comparison using overlapping segments

```
Comparing point estimates
        stochastic.m: -2.974485e-06
        pyGWB: -2.974485e-06
        % diff: 0.000001%
Comparing sigmas
        stochastic.m: 1.231366e-06
        pyGWB: 1.231366e-06
        % diff: 0.000000%
```



Comparison using overlapping segments

- For the results shown on the previous slide, we tried to use identical data and processing for both stochastic.m and pyGWB.
- However in practice there will be differences between stochastic.m and pyGWB because of the differences in implementation, such as filters etc.
- Below are the results we get with current pyGWB implementation.

```
Comparing point estimates
        stochastic.m: -2.974485e-06
        pyGWB (modifed post-process): -3.057627e-06
        % diff: 2.795157%
Comparing sigmas
        stochastic.m: 1.231366e-06
        pyGWB (modifed post-process): 1.229126e-06
        % diff: 0.181856%
```